

## Sensoroversikt (hurtigguide)

| Sensor / tema    | Hva                                                | Grensesnitt                  | Seksjon |
|------------------|----------------------------------------------------|------------------------------|---------|
| BME280           | Temp, fukt, trykk                                  | I <sup>2</sup> C / SPI       | 2       |
| PIR HC-SR501     | Bevegelse                                          | Digital inn / analog nivå    | 3       |
| OLED BME12864-11 | 128x64 skjerm                                      | I <sup>2</sup> C             | 4       |
| LDR              | Lys                                                | Analog inn (spenningsdeler)  | 5       |
| ENS160           | Luftkvalitet (TVOC/eCO <sub>2</sub> )              | I <sup>2</sup> C             | 6       |
| SEN-13637        | Jordfuktighet                                      | Analog inn                   | 7       |
| RFID RC522       | RFID-kort/tag                                      | SPI                          | 8       |
| 3x4 Keypad       | Tastatur                                           | Flere digitale inn           | 9       |
| NeoPixel         | RGB-LED-stripe                                     | Én digital ut                | 10      |
| SG90 servo       | Vinkelmotor                                        | PWM-ut                       | 11      |
| Piezo-buzzer     | Lyd (tone/pip)                                     | Digital ut (tone())          | 12      |
| 16x2 LCD         | Tekstdisplay                                       | Parallell / I <sup>2</sup> C | 13      |
| Feilsøking       | Generell sjekkliste + I <sup>2</sup> C-skanner     | –                            | 14      |
| Signalbehandling | Filtrering, glidende gjennomsnitt, hysteres        | –                            | 15      |
| Koblingstips     | Strøm, jord, I <sup>2</sup> C, SPI, analog/digital | –                            | 16      |
| State-maskin     | Eksempel på trafikklys med millis()                | –                            | 17      |

# 1 Oversikt og kategorier

Første side er en ren hurtigguide. Denne siden gir en litt mer «logisk» oversikt over kapitlene, gruppert etter type modul.

## 1.1 Miljø- og lyssensorer

**BME280** Temperatur, relativ fuktighet og lufttrykk i én chip (seksjon 2).

**ENS160** Luftkvalitet (TVOC, eCO<sub>2</sub>), ofte brukt sammen med BME280 (seksjon 6).

**LDR** Lyssensor basert på fotomotstand og spenningsdeler (seksjon 5).

**Jordfuktighet SEN-13637** Analog jordfuktighetssensor med to sonder i jorda (seksjon 7).

## 1.2 Bevegelse, nærvær og tilgang

**PIR HC-SR501** Oppdager bevegelse fra mennesker/dyr ved IR/varme, brukt både som digital og «analog» sensor (seksjon 3).

**RFID RC522** Leser RFID-/MIFARE-kort og -tagger via SPI, typisk brukt til adgangskontroll (seksjon 8).

## 1.3 Skjermer og lys

**OLED 128x64** Liten monokrom skjerm for tekst og grafikk (seksjon 4).

**16x2 LCD** Klassisk tegnbasert tekstdisplay (seksjon 13).

**NeoPixel (WS2812B)** Adressérbare RGB-LED-er, én digital pin styrer hele stripen (seksjon 10).

## 1.4 Brukerinput og lyd

**3x4 keypad** Numerisk tastatur med 7 pinner (rader/kolonner) (seksjon 9).

**Piezo-buzzer** Enkel lyd giver for pip/toner (seksjon 12).

## 1.5 Aktuatorer og motorer

**SG90 servo** Liten RC-servo for posisjonering (0–180°) (seksjon 11).

**Små DC-motorer** Kjøres via transistor eller H-bro-driver (omtalt i seksjon 16).

## 1.6 Støttekapitler

**Feilsøking** Sjekklistor, typiske feil, I<sup>2</sup>C-skanner (seksjon 14).

**Signalbehandling** Glidende gjennomsnitt, hysteres, medianfiltrering (seksjon 15).

**Koblingstips** Strøm, jord, breadboard, I<sup>2</sup>C/SPI, motorstyring (seksjon 16).

**State-maskin** Eksempel på trafikklys med ikke-blokkerende `millis()`-logikk (seksjon 17).

## 2 Fukt- og temperatursensor SparkFun BME280

### Hva gjør den?

Måler temperatur, relativ fuktighet og lufttrykk i én chip.

### Viktige tall

- Forsyning (breakout): 3.3 V
- Grensesnitt: I<sup>2</sup>C (vanligst) eller SPI
- I<sup>2</sup>C-adresser: 0x76 eller 0x77

### Tilkobling (I<sup>2</sup>C)

- **3.3V** – 3.3 V
- **GND** – jord
- **SDA** – til I<sup>2</sup>C SDA på mikrokontroller
- **SCL** – til I<sup>2</sup>C SCL på mikrokontroller

### Datablad / bibliotek

- Datablad: Bosch BME280
- Arduino-bibliotek: Adafruit\_BME280\_Arduino\_Library

## BME280 Sensor Example Code (Adafruit Library)

```
// BME280 environmental sensor example using Adafruit library (I2C)
// Measures temperature, humidity, pressure, and altitude

#include <Wire.h>
#include <Adafruit_Sensor.h>
#include <Adafruit_BME280.h>

#define SEALEVELPRESSURE_HPA 1013.25

Adafruit_BME280 bme; // I2C interface

void setup() {
  Serial.begin(115200);
  delay(100);

  // --- BME280 init (try 0x76 then 0x77) ---
  bool bmeOK = false;
  Serial.println(F("Trying BME280 at 0x76..."));
  if (bme.begin(0x76)) {
    bmeOK = true;
    Serial.println(F("BME280 found at 0x76"));
  } else {
    Serial.println(F("No BME280 at 0x76, trying 0x77..."));
    if (bme.begin(0x77)) {
      bmeOK = true;
      Serial.println(F("BME280 found at 0x77"));
    }
  }
  if (!bmeOK) {
    Serial.println(F("BME280 not found at 0x76 or 0x77, check wiring or I2C address!"));
    while (1) { delay(10); }
  }
}
```

```

}

Serial.println("Adafruit BME280 ready.");
}

void loop() {
  float temperature = bme.readTemperature();           // C
  float humidity    = bme.readHumidity();              // %RH
  float pressure    = bme.readPressure() / 100.0F;    // hPa
  float altitude    = bme.readAltitude(SEALEVELPRESSURE_HPA); // meters

  Serial.print("Temperature: ");
  Serial.print(temperature, 2);
  Serial.println(" C ");

  Serial.print("Humidity:   ");
  Serial.print(humidity, 1);
  Serial.println(" %RH");

  Serial.print("Pressure:   ");
  Serial.print(pressure, 1);
  Serial.println(" hPa");

  Serial.print("Altitude:   ");
  Serial.print(altitude, 1);
  Serial.println(" m");

  Serial.println();

  delay(1000); // read once per second
}

```

### 3 PIR-bevegelsessensor HC-SR501

#### Hva gjør den?

Oppdager bevegelse fra mennesker/dyr ved å måle infrarød (varme).

#### Viktige tall

- Forsyning: 5 V (kan ofte ta mer, men 5 V er typisk)
- Utgang: digital (HIGH  $\approx$  3.3 V, LOW  $\approx$  0)

#### Tilkobling

- **VCC** – 5 V
- **OUT** (svart) – til digital input på mikrokontroller
- **GND** (hvit) – jord

#### Datablad / dokumentasjon

- Lokalt kurs-/labark (HiØ) eller generelle HC-SR501-oversikter på nett

### PIR Sensor (Analog Read) Example Code

Noen PIR-moduler oppfører seg litt forskjellig:

- Noen går **HIGH** (høy spenning) når de ser bevegelse, og er lave ellers.
- Andre er motsatt og går **LOW** ved bevegelse.

Derfor:

- **Terskelen** (`pirThreshold`) må tilpasses din PIR ved å se på verdiene i Serial Monitor.
- Om du skal teste på `> pirThreshold` eller `< pirThreshold` avhenger av om sensoren går høy eller lav når den ser bevegelse.

Eksempelkode med `analogRead()` av PIR-signal:

```
// Interfacing Arduino Uno with PIR sensor using analogRead()
// Connect PIR output to analog pin A0

const int pirPin = A0;    // PIR output on analog pin A0
const int ledPin = 5;     // LED on digital pin 5

// Choose a threshold based on your sensor values.
// Use Serial Monitor to fine-tune (e.g. 300 600 for many modules).
int pirThreshold = 300;   // example value      adjust for your PIR

void setup() {
  Serial.begin(9600);
  pinMode(ledPin, OUTPUT); // LED as output

  Serial.println("PIR (analog) sensor starting...");
  delay(5000);             // allow sensor to stabilize
  Serial.println("Ready!");
}

void loop() {
  int pirValue = analogRead(pirPin); // read analog value 0 1023
```

```

Serial.print("PIR value: ");
Serial.println(pirValue);

// Most PIR modules go HIGH when they see motion:
// - In that case: "motion" when pirValue > pirThreshold.
// If your PIR goes LOW when it sees motion, swap > with <.

if (pirValue > pirThreshold) {      // motion detected
  digitalWrite(ledPin, HIGH);
  Serial.println("Motion detected!");
} else {                            // no motion
  digitalWrite(ledPin, LOW);
  Serial.println("No motion");
}

delay(1000); // small delay for readability
}

```

## PIR og tidsstyring med `millis()`

I enkle eksempler brukes ofte `delay()` (f.eks. `delay(1000);`) for å vente. Ulempen er at **hele programmet stopper** i denne perioden – Arduino-en gjør ingenting annet.

Med `millis()` kan vi måle tid uten å blokkere:

- `millis()` gir antall millisekunder siden Arduino-en ble startet.
- Vi kan lagre tidspunktet for «sist gjort noe» og sjekke om det har gått X ms.

Eksempelet under:

- Leser PIR med jevne mellomrom (`sampleInterval`).
- Skruer på LED når det oppdages bevegelse.
- Holder LED på i `ledOnTime` ms etter *siste* bevegelse.
- Bruker **ingen `delay()`** i `loop()`.

```
// PIR with millis(): non-blocking timing
// - Samples PIR every sampleInterval ms
// - Turns LED on when motion is detected
// - Keeps LED on for ledOnTime ms after last motion

const int pirPin  = A0; // PIR output to analog pin A0
const int ledPin  = 5;  // LED on digital pin 5

// Threshold must be tuned for your PIR and wiring:
// - Use Serial Monitor to see typical values with/without motion.
// - For most PIRs that go HIGH on motion, we test pirValue > pirThreshold.
int pirThreshold = 300; // example value      adjust!

// Timing parameters (in milliseconds)
const unsigned long sampleInterval = 100;    // how often we read PIR
const unsigned long ledOnTime      = 5000;    // keep LED on 5 s after last motion

// State variables
unsigned long lastSampleTime = 0; // last time we read PIR
unsigned long lastMotionTime = 0; // last time motion was detected
bool          motionDetected = false;

void setup() {
  Serial.begin(9600);
  pinMode(ledPin, OUTPUT);
  digitalWrite(ledPin, LOW);

  Serial.println("PIR (analog) with millis() starting...");
  delay(5000); // one-time warmup for the PIR
  Serial.println("Ready!");
}

void loop() {
  unsigned long now = millis();

  // 1) Sample PIR at fixed intervals (non-blocking)
  if (now - lastSampleTime >= sampleInterval) {
    lastSampleTime = now;

    int pirValue = analogRead(pirPin); // 0 1023

    Serial.print("PIR value: ");
    Serial.println(pirValue);

    // If your PIR goes LOW on motion, change '>' to '<' here.
    if (pirValue > pirThreshold) {
```

```

    motionDetected = true;
    lastMotionTime = now; // remember when we last saw motion
    Serial.println("Motion detected!");
} else {
    motionDetected = false;
    Serial.println("No motion");
}
}

// 2) LED control: stay ON for ledOnTime ms after last motion
if (now - lastMotionTime <= ledOnTime) {
    // recently saw motion -> LED ON
    digitalWrite(ledPin, HIGH);
} else {
    // no motion for a while -> LED OFF
    digitalWrite(ledPin, LOW);
}

// No delay() needed here; loop() can run freely and do other tasks.
}

```

## PIR (digital) – pass på riktig kobling (VCC/GND)

```

#define ledPin 5
#define pirPin 8

int pirValue = 0;
int pirState = LOW;

void setup() {
    Serial.begin(9600);
    pinMode(ledPin, OUTPUT);
    pinMode(pirPin, INPUT_PULLUP);
    Serial.println("PIR Sensor ready!");
    delay(5000);
}

void loop() {
    int motion = digitalRead(pirPin);

    if (motion == HIGH) {
        if (pirState == LOW) {
            Serial.println("Motion Detected");
            digitalWrite(ledPin, HIGH);
            pirState = HIGH;
        }
    } else {
        if (pirState == HIGH) {
            Serial.println("Not Detected");
            digitalWrite(ledPin, LOW);
            pirState = LOW;
        }
    }

    delay(500);
}

```

### Merk:

- Terskelverdien **pirThreshold** i de analoge eksemplene avhenger av PIR-en: bruk Serial Monitor for å se typiske verdier med og uten bevegelse, og juster terskelen deretter.
- Om PIR-en går **HIGH** eller **LOW** ved bevegelse avhenger også av modul og oppsett:
  - Går den **HIGH** ved bevegelse, bruk test som i eksempelet: **pirValue > pirThreshold**.



- Går den **LOW** ved bevegelse, bytt til  $\text{pirValue} < \text{pirThreshold}$ .

## 4 OLED-skjerm BME12864-11

### Hva gjør den?

Liten 128x64 monokrom OLED-skjerm for tekst og enkel grafikk.

### Viktige tall

- Forsyning: ca. 3.3 V
- Grensesnitt: I<sup>2</sup>C

### Tilkobling (I<sup>2</sup>C)

- VDD – 3.3 V
- GND – jord
- SCL/SCK – I<sup>2</sup>C-klokke
- SDA – I<sup>2</sup>C-data

### Datablad / dokumentasjon

- GME12864-11 Product Specification
- Adafruit SSD1306-dokumentasjon

### Bibliotek

- Adafruit\_SSD1306
- Adafruit\_GFX

## OLED Display Example Code

```
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

#define SCREEN_WIDTH 128 // OLED display width, in pixels
#define SCREEN_HEIGHT 64 // OLED display height, in pixels

// I2C OLEDs usually don't need reset pin; use -1
#define OLED_RESET -1
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET);

void setup() {
  Serial.begin(9600);
  if (!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) { // 0x3C is a common I2C address
    Serial.println(F("SSD1306 allocation failed"));
    for (;;); // Don't proceed, loop forever
  }
  display.clearDisplay();
  display.setTextSize(2);           // Text size
  display.setTextColor(WHITE);     // Text color
  display.setCursor(0, 0);          // Top-left corner
  display.println("Hello,");
  display.println("OLED!");
  display.display();                // Show text
}
```

```
void loop() {  
  // Keep the message on screen  
}
```

## 5 LDR – Light Dependent Resistor (lyssensor)

### Hva gjør den?

Motstand som endrer verdi med lysnivå. Mørkt = høy motstand, lyst = lav motstand.

### Tilkobling (spenningsdeler)

- Koble LDR i serie med en fast motstand  $R_{\text{fast}}$ .
- Ene enden til  $V_{\text{CC}}$ , andre enden til GND.
- Les spenningen i midtpunktet (mellom LDR og  $R_{\text{fast}}$ ) på en analog inngang.

### Tips

- Velg  $R_{\text{fast}}$  i samme størrelsesorden som LDR-motstanden ved «midt på treet»-lys.

### Datablad / dokumentasjon (eksempel)

- Eksempel-datablad for CdS-fotomotstand

## LDR Sensor Example Code

```
// Interfacing Arduino uno with LDR sensor
// LDR connection: connect one leg to power, the other leg goes to analog pin
// and ground through a 10k resistor.

const int ledPin = 5; // digital pin 5
const int ldrPin = A0; // analog pin 0

void setup() {
  // Void setup function will only run once, after each powerup or reset.
  Serial.begin(9600);
  pinMode(ledPin, OUTPUT); // LED as output/indicator
  pinMode(ldrPin, INPUT); // LDR as input (via spenningsdeler)
}

void loop() { // loop() runs again and again

  int ldrStatus = analogRead(ldrPin);

  if (ldrStatus <= 200) {
    digitalWrite(ledPin, HIGH); // Darkness -> LED on
    Serial.println("Darkness over here, turn on the LED : ");
    Serial.println(ldrStatus);
  } else {
    digitalWrite(ledPin, LOW); // Sufficient light -> LED off
    Serial.println("There is sufficient light, turn off the LED : ");
    Serial.println(ldrStatus);
  }
}
```

## 6 Luftkvalitetssensor ENS160

### Hva gjør den?

Måler luftkvalitet (TVOC, eCO<sub>2</sub>) og gir digitale verdier via I<sup>2</sup>C.

### Viktige tall (breakout)

- Forsyning: 3.3 V–5 V (på typisk breakout)
- Grensesnitt: I<sup>2</sup>C (ofte adresse 0x53)
- Krever litt oppvarmingstid før stabile målinger

### Tilkobling (I<sup>2</sup>C)

- VCC – 3.3 V–5 V
- GND – jord
- SDA – I<sup>2</sup>C-data
- SCL – I<sup>2</sup>C-klokke

### Datablad / bibliotek

- Datablad: ENS160 datasheet
- Arduino: SparkFun ENS160 Arduino Library
- Tutorial: Qwiic hookup guide

## ENS160 + BME280 Combined Example Code

```
#include <Wire.h>

#include <Adafruit_Sensor.h>
#include <Adafruit_BME280.h>
#include <SparkFun_ENS160.h>

SparkFun_ENS160 myENS;      // ENS160 (SparkFun library)
Adafruit_BME280 bme;       // BME280 (Adafruit library)

void setup() {
  Serial.begin(115200);
  delay(2000);             // give USB serial time to come up
  Wire.begin();            // for Uno/Nano: SDA=A4, SCL=A5

  Serial.println(F("Starting ENS160 + BME280 test"));

  // --- ENS160 init ---
  if (!myENS.begin()) {
    Serial.println(F("ENS160 not found, check wiring!"));
    while (1) { delay(10); }
  }
  Serial.println(F("ENS160 OK"));

  // --- BME280 init (try 0x76 then 0x77) ---
  bool bmeOK = false;

  Serial.println(F("Trying BME280 at 0x76..."));
  if (bme.begin(0x76)) {
    bmeOK = true;
  }
```

```

    Serial.println(F("BME280 found at 0x76"));
} else {
    Serial.println(F("No BME280 at 0x76, trying 0x77..."));
    if (bme.begin(0x77)) {
        bmeOK = true;
        Serial.println(F("BME280 found at 0x77"));
    }
}

if (!bmeOK) {
    Serial.println(F("BME280 not found at 0x76 or 0x77, check wiring or I2C address!"));
    while (1) { delay(10); }
}

// --- ENS160 mode ---
myENS.setOperatingMode(SFE_ENS160_RESET);
delay(100);
myENS.setOperatingMode(SFE_ENS160_STANDARD);

Serial.println(F("ENS160 + Adafruit BME280 started."));
}

void loop() {
    // Read environment from BME280 (Adafruit)
    float tempC    = bme.readTemperature(); // C
    float humidity = bme.readHumidity();     // %RH
    float pressure = bme.readPressure();     // Pa

    // Give compensation data to ENS160
    myENS.setTempCompensationCelsius(tempC);
    myENS.setRHCompensationFloat(humidity);

    // When ENS160 has new data, print everything
    if (myENS.checkDataStatus()) {
        Serial.println(F("---- BME280 (Adafruit) ----"));
        Serial.print(F("Temp: "));
        Serial.print(tempC, 2);
        Serial.println(F(" C "));

        Serial.print(F("Humidity: "));
        Serial.print(humidity, 1);
        Serial.println(F(" %RH"));

        Serial.print(F("Pressure: "));
        Serial.print(pressure, 0);
        Serial.println(F(" Pa"));

        Serial.println(F("---- ENS160 (SparkFun) ----"));
        Serial.print(F("AQI: "));
        Serial.println(myENS.getAQI());

        Serial.print(F("TVOC: "));
        Serial.print(myENS.getTVOC());
        Serial.println(F(" ppb"));

        Serial.print(F("eCO2: "));
        Serial.print(myENS.getECO2());
        Serial.println(F(" ppm"));

        Serial.println();
    }

    delay(500);
}

```

## 7 Jordfuktighetssensor SparkFun SEN-13637

### Hva gjør den?

Måler hvor fuktig jorda er via motstand mellom to sonder. Gir analog spenning.

### Viktige tall

- Forsyning: 3.3 V–5 V
- Grensesnitt: analog utgang (**SIG**)

### Tilkobling

- **VCC** – 3.3 V–5 V
- **GND** – jord
- **SIG** – til analog inngang

### Tips

- Slå gjerne sensoren av med en digital pin, og bare slå på når du måler (for mindre korrosjon).

### Datablad / dokumentasjon

- SparkFun Soil Moisture Sensor Hookup Guide
- SparkFun SEN-13637 produktside

## Soil Moisture Sensor Example Code

```
// Enkel jordfuktighetsmåling med SparkFun SEN-13637
// SIG kobles til analog inngang A0.
// Juster terskelverdien etter kalibrering.

const int soilPin = A0; // analog inngang til SIG
const int ledPin = 5;   // valgfritt: LED eller rele for pumpe

// Eksempel terskel: lav verdi = våt jord, høy verdi = tørr jord.
// MÅ kalibreres i deres oppsett (se serielldata).
const int dryThreshold = 600;

void setup() {
  Serial.begin(9600);
  pinMode(ledPin, OUTPUT);
  Serial.println("Jordfuktighetssensor klar.");
}

void loop() {
  int raw = analogRead(soilPin); // 0-1023

  Serial.print("Rå jordfuktighetsverdi: ");
  Serial.println(raw);

  // Enkel "vann nå?"-logikk:
  if (raw > dryThreshold) {
    Serial.println("Tørt! Slår på LED/pumpe.");
    digitalWrite(ledPin, HIGH);
  } else {
    Serial.println("Nok fukt. Slår av LED/pumpe.");
    digitalWrite(ledPin, LOW);
  }
}
```

```
}  
  
Serial.println();  
delay(500);  
}
```



## 8 RFID-leser RC522 (MFRC522)

### Hva gjør den?

Leser og skriver til RFID/MIFARE-kort og -brikker på 13.56 MHz via SPI.

### Viktige tall

- Forsyning: 3.3 V
- Grensesnitt: SPI

### Tilkobling (typisk mot Arduino Uno, SPI)

- VCC – 3.3 V
- GND – jord
- SCK – D13
- MOSI – D11
- MISO – D12
- SDA/SS – D10 (chip select)
- RST – f.eks. D9

### Datablad / dokumentasjon

- NXP MFRC522 datasheet

### Bibliotek

- miguelbalboa/rfid (Arduino-bibliotek)

## RFID RC522 Simple UID Read Example

```
// Enkel RFID-leser med RC522 (MFRC522)
// Leser UID fra kort/tag og skriver til Serial Monitor.
//
// Koble til som beskrevet i teksten:
// SDA/SS -> D10, SCK -> D13, MOSI -> D11, MISO -> D12, RST -> D9

#include <SPI.h>
#include <MFRC522.h>

const byte SS_PIN = 10;
const byte RST_PIN = 9;

MFRC522 rfid(SS_PIN, RST_PIN);

void setup() {
  Serial.begin(115200);
  SPI.begin();           // start SPI
  rfid.PCD_Init();       // start MFRC522

  Serial.println(F("RC522 klar. Hold et kort/tag inntil leseren."));
}

void loop() {
  // Sjekk om det er et nytt kort i nærheten
```

```

if (!rfid.PICC_IsNewCardPresent()) {
    return;
}

// Forsøk å lese UID
if (!rfid.PICC_ReadCardSerial()) {
    return;
}

Serial.print(F("UID: "));
for (byte i = 0; i < rfid.uid.size; i++) {
    if (rfid.uid.uidByte[i] < 0x10) {
        Serial.print("0");
    }
    Serial.print(rfid.uid.uidByte[i], HEX);
    Serial.print(" ");
}
Serial.println();

// Valgfritt: vis korttype
MFRC522::PICC_Type piccType = rfid.PICC_GetType(rfid.uid.sak);
Serial.print(F("Korttype: "));
Serial.println(rfid.PICC_GetTypeName(piccType));

Serial.println();

// Stopp kommunikasjon med kortet
rfid.PICC_HaltA();
rfid.PCD_StopCrypto1();
}

```

## 9 3x4 matrise-keypad

### Hva gjør den?

Lite numerisk tastatur (12 taster): 0–9, \* og #.

### Viktige tall

- 12 taster (3 kolonner x 4 rader)
- 7 tilkoblingspinner: 4 rader + 3 kolonner

### Tilkobling

- Koble alle 7 pinnene til digitale I/O-pinner på mikrokontrolleren.
- Rekkefølgen (hvilken pinne som er ROW/COL) sjekkes i datablad eller med multimeter.

### Datablad / dokumentasjon (eksempel-modul)

- Typisk 3x4 Keypad-modul (pinout/dokumentasjon)

### Bibliotek

- Keypad-bibliotek for Arduino

## 10 NeoPixel (WS2812B adressérbare RGB-LED-er)

### Hva gjør den?

Adressérbare RGB-LED-er der hver LED styres individuelt via én datalinje.

### Viktige tall

- Forsyning: 5 V
- Grensesnitt: éntårds seriedata fra digital pin

### Tilkobling

- **5V** – 5 V (dimensjoner strøm etter antall LED-er)
- **GND** – felles jord med mikrokontroller
- **DIN** – til digital utgang.

### Datablad / dokumentasjon

- WS2812B datasheet

### Bibliotek

- Adafruit\_NeoPixel
- Alternativ: FastLED

## NeoPixel (Adafruit) Simple Example

```
// Simple Adafruit NeoPixel example
// Turns the whole strip red, then green, then blue.

#include <Adafruit_NeoPixel.h>

#define LED_PIN    6    // Data pin for the NeoPixel strip
#define LED_COUNT  8    // Number of LEDs in the strip

Adafruit_NeoPixel strip(LED_COUNT, LED_PIN, NEO_GRB + NEO_KHZ800);

void setup() {
  strip.begin();           // Initialize the strip
  strip.setBrightness(50); // Set brightness (0 255 )
  strip.show();            // Turn all LEDs off at start
}

// Helper function: fill all LEDs with one color
void fillStrip(uint8_t r, uint8_t g, uint8_t b) {
  for (int i = 0; i < LED_COUNT; i++) {
    strip.setPixelColor(i, strip.Color(r, g, b)); // set color for pixel i
  }
  strip.show(); // send the updated colors to the strip
}

void loop() {
  fillStrip(255, 0, 0); // Red
  delay(500);

  fillStrip(0, 255, 0); // Green
  delay(500);
```

```
    fillStrip(0, 0, 255); // Blue  
    delay(500);  
}
```

## 11 Servomotor SG90 (RC-servo)

### Hva gjør den?

Liten motor som kan stilles til en bestemt vinkel (f.eks. 0–180°).

### Viktige tall

- Forsyning: ca. 5 V
- Styring: PWM-signal, ca. 50 Hz

### Tilkobling

- Rød: 5 V
- Brun/sort: GND
- Oransje/gul: styresignal til PWM-pin på mikrokontroller

### Datablad / dokumentasjon

- Eksempel-datablad SG90 micro servo

### Bibliotek

- Arduino Servo-bibliotek

## Servo Example Code

```
// Include the Servo library
#include <Servo.h>

Servo myservo; // create servo object to control a servo
// twelve servo objects can be created on most boards

int pos = 0;    // variable to store the servo position

void setup() {
  myservo.attach(9); // attaches the servo on pin 9 to the servo object
}

void loop() {
  for (pos = 0; pos <= 180; pos += 1) { // goes from 0 degrees to 180 degrees
    // in steps of 1 degree
    myservo.write(pos);              // tell servo to go to position in variable 'pos'
    delay(15);                       // waits 15ms for the servo to reach the position
  }
  for (pos = 180; pos >= 0; pos -= 1) { // goes from 180 degrees to 0 degrees
    myservo.write(pos);              // tell servo to go to position in variable 'pos'
    delay(15);                       // waits 15ms for the servo to reach the position
  }
}
```

## 12 Piezo-buzzer

### Hva gjør den?

Liten «høytaler» som kan lage pipelyder og enkle toner når den drives fra en digital pinne (gjern med `tone()`-funksjonen).

### Viktige tall

- Forsyning: 3.3 V–5 V (avhengig av modul/buzzer)
- Styring: digital pin (PWM/`tone()`)

### Tilkobling

- + / lang pinne – til digital utgang (ofte via en liten seriemotstand, f.eks. 100  $\Omega$ )
- - / kort pinne – til GND

### Tips

- Bruk `noTone(pin)`; for å stoppe lyden.
- En «passiv» piezo-buzzer trenger frekvens (f.eks. 1000 Hz); en «aktiv» buzzer lager lyd bare ved HIGH/LOW.

## Piezo Buzzer Example Code

```
// Simple Arduino piezo buzzer example
// Plays a short "beep-beep" pattern on a passive buzzer

const int buzzerPin = 8; // connect buzzer + to digital pin 8, - to GND

void setup() {
  // no setup needed
}

void loop() {
  // First beep
  tone(buzzerPin, 1000); // 1000 Hz (1 kHz)
  delay(200);           // play for 200 ms
  noTone(buzzerPin);    // stop sound
  delay(100);           // short pause

  // Second beep (higher pitch)
  tone(buzzerPin, 1500); // 1500 Hz
  delay(200);
  noTone(buzzerPin);

  // Wait a bit before repeating
  delay(1000);
}
```

## 13 16x2 LCD med HD44780-kontroller

### Hva gjør den?

Tekstdisplay med 2 linjer x 16 tegn. HD44780-kompatibel kontroller.

### Viktige tall

- Forsyning (logikk): 5 V
- Grensesnitt: 4-bit parallell (vanligst) eller via I<sup>2</sup>C-bakplate

### Tilkobling (4-bit parallell, kortversjon)

- VSS – GND
- VDD – 5 V
- VO – kontrast (potmeter mellom 5 V og GND)
- RS, E – til digitale pinner
- R/W – til GND (skrivemodus)
- D4–D7 – til digitale pinner (data)

### Datablad / dokumentasjon

- HD44780U kontroller-datasheet

### Bibliotek

- Arduino LiquidCrystal-bibliotek

## 16x2 LCD "Hello, world!Example Code

```
// Enkel 16x2 LCD "Hello, world!" med HD44780-kontroller
// 4-bit parallell grensesnitt (uten I2C-bakplate).

#include <LiquidCrystal.h>

// RS, E, D4, D5, D6, D7 (tilpass pinnene til deres oppkobling)
LiquidCrystal lcd(12, 11, 5, 4, 3, 2);

void setup() {
  lcd.begin(16, 2);           // 16 kolonner, 2 rader
  lcd.print("Hello, world!"); // første linje

  // Skriv noe på linje 2:
  lcd.setCursor(0, 1);        // kolonne 0, rad 1
  lcd.print("LCD demo");
}

void loop() {
  // Eksempel: oppdater en teller i hjørnet
  static unsigned long lastUpdate = 0;
  static int counter = 0;

  unsigned long now = millis();
  if (now - lastUpdate > 1000) { // hver 1 sekund
    lastUpdate = now;
```



```
    lcd.setCursor(11, 1);          // høyre side på linje 2
    lcd.print(" ");                // "slett" tidligere tall
    lcd.setCursor(11, 1);
    lcd.print(counter);
    counter++;
  }
}
```

## 14 Feilsøking

### Generell sjekkliste

Når «ingenting virker», sjekk alltid dette først:

- **Strøm:** Får sensoren riktig forsyningsspenning? (3.3 V vs. 5 V)
- **GND felles:** Mikrokontroller og alle moduler *må* ha felles jord.
- **Riktig pinne:**
  - Stemmer pinnen i koden med faktisk tilkobling på brettet?
  - For PWM: bruk en pinne som faktisk støtter PWM.
- **Bibliotek:** Er riktig bibliotek installert og inkludert (`#include`)?
- **Eksempel først:** Start med et *enkelt* eksempel (f.eks. «Hello, world!» på LCD) før du blander flere sensorer.
- **Seriell monitor:** Bruk `Serial.print()` for å se hva som faktisk skjer.
- **Oppvarmingstid:** Noen sensorer (PIR, ENS160) trenger noen sekunder til minutter før de gir stabile verdier.

### Typiske symptomer og årsaker

- **Ingen liv i sensor (helt dødt):**
  - Feil spenning, eller VCC/GND byttet.
  - Kabel ikke i, brudd i breadboard-rad.
  - Sjekk dioden om den lyser.
  - Bruk multimeter for å sjekke spenning/strøm.
- **Merkelig/noisy analogverdi (LDR, jordfuktighet):**
  - Dårlig kontakt, lange ledninger, elektromagnetisk støy.
  - Manglende eller feil dimensjonert motstand i spenningsdeleren.
  - Manglende felles jord.
- **Servo/NeoPixel får Arduino til å resettes:**
  - For mye strøm fra 5 V-pinnen på Arduino.
  - Løsning: bruk egen 5 V-forsyning, men *felles GND*.

### I<sup>2</sup>C-feilsøking

I<sup>2</sup>C brukes av blant annet BME280, ENS160 og OLED-skjermen. Hvis de ikke svarer:

- Sjekk at SDA/SCL er koblet til *riktige* pinner på mikrokontrolleren.
- Sjekk at alle enheter har felles GND.
- Sjekk at adressene stemmer:
  - BME280: `0x76` eller `0x77`
  - ENS160: typisk `0x53`
  - OLED (SSD1306): ofte `0x3C` eller `0x3D`
- På mange breakout-kort er pull-up-motstandene på SDA/SCL allerede innebygd. Hvis du bruker *rå* chip eller spesielle moduler, kan du trenge egne pull-ups (typisk 4.7 k $\Omega$  til 10 k $\Omega$  til 3.3 V eller 5 V).

## I<sup>2</sup>C-skanner (Arduino-kode)

Bruk denne koden for å se hvilke I<sup>2</sup>C-enheter som faktisk svarer på bussen. *Fremgangsmåte:* Koble opp I<sup>2</sup>C-enheten(e), last opp koden, åpne Seriell monitor.

```
// Enkel I2C-skanner for Arduino
// Viser alle adresser (0x01 - 0x7E) som svarer på I2C-bussen.
```

```
#include <Wire.h>

void setup() {
    Wire.begin();           // start I2C (SDA/SCL)
    Serial.begin(115200);

    // Vent til seriell er klar (nyttig på noen kort)
    while (!Serial) {
        ; // gjør ingenting
    }

    Serial.println("I2C-skanner starter...");
}

void loop() {
    byte error;
    byte address;
    int nDevices = 0;

    Serial.println("Skanner adresser 0x01 til 0x7E...");

    for (address = 1; address < 127; address++) {
        // Start en transmisjon til denne adressen:
        Wire.beginTransmission(address);
        error = Wire.endTransmission();

        if (error == 0) {
            Serial.print("Fant enhet på adresse 0x");
            if (address < 16) Serial.print("0");
            Serial.println(address, HEX);
            nDevices++;
        } else if (error == 4) {
            Serial.print("Ukjent feil på adresse 0x");
            if (address < 16) Serial.print("0");
            Serial.println(address, HEX);
        }
    }

    if (nDevices == 0) {
        Serial.println("Ingen I2C-enheter funnet.\n");
    } else {
        Serial.println("Skann ferdig.\n");
    }

    delay(2000); // vent litt før neste skann
}
```

## Tolkning av I<sup>2</sup>C-skann

- **Ingen enheter funnet:**
  - Feil SDA/SCL-pinner brukt.
  - Manglende felles GND.
  - Ingen strøm til sensoren.
- **Adresse som ikke stemmer med det du forventer:**
  - Noen breakouts lar deg endre adresse med en jumper/lodding.

- Juster adressen i koden til den du ser i skanneren.
- **Flere enheter på samme adresse:**
  - I<sup>2</sup>C krever unik adresse per enhet.
  - Flytt én av dem til en annen adresse (jumper) eller bytt til SPI/annen sensor.

## 15 Signalbehandling av sensordata

### Hvorfor signalbehandling?

Rå sensordata er ofte:

- **støyete** (verdiene hopper litt opp og ned)
- har **spikes** (enkeltmålinger som er helt på jordet)
- ligger rundt en **terskel** slik at ting blinker/oscillerer

Enkle grep i programvaren kan gjøre signalene mye mer brukbare, uten ekstra elektronikk.

Typiske teknikker:

- **Glidende gjennomsnitt** (moving average) – gjør signalet «roligere».
- **Medianfilter** – fjerner enkelt-spikes.
- **Hysteres** – hindrer at noe slår av/på hele tiden rundt en terskel.

Nedenfor er enkle eksempler som kan brukes på f.eks. LDR, jordfuktighet eller andre analoge sensorer.

### Glidende gjennomsnitt (moving average)

Ideen: i stedet for å bruke *siste måling*, bruker vi *snittet* av de N siste målingene.

```
// Leser en analog sensor (f.eks. LDR eller jordfuktighet) på A0
// og glatter verdien med et glidende gjennomsnitt.
```

```
const int sensorPin = A0;
const int ledPin     = 5;

// Antall målinger i snittet (jo høyere, jo glattere men tregere)
const int N = 10;

int  buffer[N];    // lagrer de N siste målingene
int  indexBuffer = 0;
long sum = 0;      // sum av verdiene i buffer

// terskel for "på/av" etter filtrering:
const int threshold = 500;

void setup() {
  Serial.begin(9600);
  pinMode(ledPin, OUTPUT);

  // initialiser buffer med første verdi
  int startVal = analogRead(sensorPin);
  for (int i = 0; i < N; i++) {
    buffer[i] = startVal;
    sum += startVal;
  }
}

void loop() {
  int raw = analogRead(sensorPin);

  // fjern eldste verdi fra sum og legg til ny
  sum -= buffer[indexBuffer];
  buffer[indexBuffer] = raw;
  sum += raw;

  indexBuffer++;
  if (indexBuffer >= N) {
    indexBuffer = 0;
  }
}
```

```

}

// glattet verdi
int filtered = sum / N;

Serial.print("Rå verdi: ");
Serial.print(raw);
Serial.print("    Filtrert verdi: ");
Serial.println(filtered);

// enkel styring basert på filtrert verdi
if (filtered > threshold) {
    digitalWrite(ledPin, HIGH);
} else {
    digitalWrite(ledPin, LOW);
}

delay(100);
}

```

## Hysteresis (unngå flimring rundt terskel)

Hvis sensoren ligger rundt en terskel, kan output flimre: HIGH, LOW, HIGH, LOW, .... Løsning: bruk to terskeler – én for *på*, en annen for *av*.

```

// Viser hysteresis på en filtrert sensormåling.
// F.eks. brukt sammen med glidende gjennomsnitt fra koden over.

// Her antar vi at "filtered" er en sensormåling mellom 0 og 1023.

const int onThreshold = 600; // slå PÅ når vi går over denne
const int offThreshold = 500; // slå AV når vi går under denne

bool isOn = false; // gjeldende tilstand

void updateOutputWithHysteresis(int filtered) {
    if (!isOn && filtered > onThreshold) {
        isOn = true;
        Serial.println("Går til: PÅ");
        // f.eks: digitalWrite(ledPin, HIGH);
    }
    else if (isOn && filtered < offThreshold) {
        isOn = false;
        Serial.println("Går til: AV");
        // f.eks: digitalWrite(ledPin, LOW);
    }

    // Hvis vi ligger mellom onThreshold og offThreshold
    // beholder vi forrige tilstand (isOn endres ikke).
}

```

## Enkel «median av 3» for å fjerne spikes

Medianfilter tar «midtverdien» av et lite sett målinger. En veldig enkel variant er «median av 3»:

```

// Median av 3 verdier: sorterer ikke hele lister, bare finner "midtverdien".
int median3(int a, int b, int c) {
    if ((a <= b && b <= c) || (c <= b && b <= a)) {
        return b;
    } else if ((b <= a && a <= c) || (c <= a && a <= b)) {
        return a;
    } else {
        return c;
    }
}

```

```

void loop() {
  int v1 = analogRead(sensorPin);
  delay(5);
  int v2 = analogRead(sensorPin);
  delay(5);
  int v3 = analogRead(sensorPin);

  int med = median3(v1, v2, v3);

  Serial.print("v1 = "); Serial.print(v1);
  Serial.print(" v2 = "); Serial.print(v2);
  Serial.print(" v3 = "); Serial.print(v3);
  Serial.print(" median = "); Serial.println(med);

  // med kan brukes videre, evt. kombinert med glidende gjennomsnitt
  delay(50);
}

```

## Datablad / dokumentasjon

- Mer om signalbehandling

## 16 Generelle koblingstips

Denne seksjonen samler noen korte, praktiske tips for å koble opp modulene i dette kompendiet: BME280, PIR, OLED, LDR, ENS160, jordfukt (SEN-13637), RFID RC522, keypad, NeoPixel, SG90, piezo-buzzer og 16x2 LCD.

### Strøm og jord

- **Felles jord:** Mikrokontroller (Arduino e.l.) og alle sensorer *må* ha felles GND.
- **Riktig spenning:**
  - Mange av modulene i oppsettet er laget for 5 V (PIR, SG90, 16x2 LCD, NeoPixel).
  - Noen sensorer er 3.3 V (typisk BME280, ENS160, OLED-moduler). Det er viktig å sjekke spesifiseringen til akkurat det kortet som brukes.
- **Strømbegrensning:** Servoer (SG90) og andre «tunge» komponenter kan trekke relativt mye strøm. Ved større oppsett bør det tas hensyn til dette og eventuelt brukes egen strømforsyning.

### Breadboard og ledninger

- **Strøm-skinner:** På en del breadboard er +/—skinnene delt på midten. Dette bør kontrolleres med multimeter, eller ved å koble en kort ledning over midtledningen.
- **Ledningslengde:** Analoge signaler (LDR, jordfukt, PIR) og I<sup>2</sup>C/SPI-signaler blir ofte penere dersom ledningene holdes relativt korte.
- **Fargekoding:** F.eks. rød = VCC, svart/blå = GND, andre farger til signal. Dette gjør feilsøking betydelig enklere.

### Digitale inn- og utganger (keypad, piezo, enkelte moduler)

- **Digital inn** (f.eks. keypad-rader/kolonner):
  - Bruk `pinMode(pin, INPUT_PULLUP)` der det trengs en definert hviletilstand.
  - Les verdier med `digitalRead()` for å se om en knapp er trykket.
- **Digital ut** (f.eks. piezo-buzzer, keypad-kolonner):
  - Vanlig `digitalWrite(HIGH/LOW)` er tilstrekkelig for de fleste små laster.
  - For piezo-buzzer kan `tone()` på en digital pinne brukes til å generere lyd.

### PIR brukt som «analog» sensor

HC-SR501-PIR-en har egentlig en digital utgang (HIGH/LOW), men i dette opplegget brukes den ofte som om den var en analog sensor:

- PIR-utgangen kobles til en **analog inngang** (f.eks. A0).
- Verdien leses med `analogRead()` for å se nivået på signalet, ikke bare 0/1.
- I koden settes en terskel (f.eks. `pirThreshold`) som skiller «ingen bevegelse» fra «bevegelse».
- På denne måten kan terskelverdien justeres underveis ved å studere målingene i Serial Monitor.

Konkrete kodeeksempler finnes i PIR-kapittelet.



## PWM og tidskritiske signaler (servo, NeoPixel, buzzer)

- **Servo (SG90):**

- En PWM-kanal digital pin (f.eks. D9 på Arduino Uno) brukes sammen med **Servo**-biblioteket.
- Servoer kan trekke en del strøm og skape støy på forsyningen, noe det bør tas høyde for når flere komponenter er koblet til samtidig.

- **NeoPixel (WS2812B):**

- Hele stripen styres fra én digital pin (f.eks. D6).
- 5V og GND på stripen kobles til 5V og GND i resten av oppsettet.
- **Viktig:** NeoPixel-stripe og mikrokontroller må ha felles GND, ellers vil stripen ikke reagere som forventet.

- **Piezo-buzzer:**

- Kan kobles direkte mellom en digital pin og GND (ofte med en liten seriemotstand, f.eks. 100  $\Omega$ ).

## Analoge signaler (LDR, jordfuktighet, PIR brukt analogt)

- **Spenningsdeler:**

- LDR kobles i serie med en fast motstand (f.eks. 10 k $\Omega$ ) som en spenningsdeler.
- Jordfukt-sensoren SEN-13637 har ferdig elektronikk på kortet og gir analog spenning på SIG.

- **Tilkobling:**

- Midtpunktet i spenningsdeleren (LDR) eller SIG fra jordfukt kobles til en analog inngang (A0, A1, ...).
- PIR-utgangen kan også kobles til en analog inngang (som i eksempelkode i PIR-kapittelet) når terskelverdier skal tunes.
- Verdien leses med `analogRead()`, og i flere eksempler brukes enkel signalbehandling (glidende gjennomsnitt / hysteresis) for å få roligere og mer stabile verdier.

## I<sup>2</sup>C (BME280, ENS160, OLED, ev. I<sup>2</sup>C-LCD)

- **Felles buss:**

- SDA og SCL er felles for alle I<sup>2</sup>C-enhetene.
- Hver enhet har sin egen adresse (BME280: 0x76/0x77, ENS160: typisk 0x53, OLED: ofte 0x3C).

- **Kobling:**

- Alle SDA-pinner kobles sammen, og alle SCL-pinner kobles sammen.
- Felles GND mellom alle enheter og mikrokontroller er helt nødvendig.

- **Pull-up-motstander:**

- De fleste breakout-kort som brukes her har innebygde pull-ups på SDA/SCL.
- Dersom det bygges «rått» uten breakout, er typiske verdier 4.7 k $\Omega$ –10 k $\Omega$  fra SDA og SCL til VCC.

## SPI (RFID-leser RC522)

- **Felles linjer:**

- MISO, MOSI og SCK deles mellom alle SPI-enheter på samme buss.
- Hver enhet får sin egen **SS/CS**-pin (chip select).

- **Typisk mot Arduino Uno:**

- SCK → D13

- MOSI → D11
- MISO → D12
- SS (SDA på RC522) → valgfri digital pin (f.eks. D10)
- RST → valgfri digital pin (f.eks. D9)
- **Mer enn én SPI-enhet:**
  - Hver enhet bør ha sin egen SS/CS-pin.
  - Bare én SS/CS skal være LOW om gangen.

## LCD (16x2) og keypad

- **16x2 LCD (parallell):**
  - 4-bit modus brukes med RS, E og D4–D7 koblet til digitale pinner.
  - VO (kontrast) styres gjerne med et lite potmeter mellom 5V og GND.
  - R/W kobles ofte direkte til GND (kun skriving).
- **Keypad (3x4):**
  - Keypaden har 7 pinner totalt: 4 rader + 3 kolonner til digitale pinner.
  - Keypad-biblioteket kan brukes til å definere hvilke Arduino-pinner som er rader/kolonner og til å lese tastetrykk.

## Små DC-motorer med NPN-transistor

Små DC-motorer (små vifter, mekaniske komponenter osv.) kan ikke kobles direkte til en Arduino-pin. En typisk løsning er å bruke en NPN-transistor som «bryter» på GND-siden:

- Motor + kobles til passende DC-forsyning (f.eks. 5 V).
- Motor – kobles til **kollektor** på NPN-transistoren.
- **Emitter** kobles til GND.
- **Base** kobles til en digital pin på Arduino via en motstand (f.eks. 1 kΩ).
- GND på motor-forsyning og Arduino kobles sammen (felles GND).

Selve på/av-styringen i koden er da enkelt: `digitalWrite(pin, HIGH/LOW);` styrer transistoren, mens transistoren håndterer motorstrømmen.

## H-bro motordriver (f.eks. L293D/L298N)

For mer avansert styring av DC-motorer (for eksempel *retning* og *hastighet*) kan en H-bro-basert motordriver brukes. Et typisk oppsett har:

- **VCC1:** logikkforsyning (f.eks. 5 V fra Arduino)
- **VCC2:** motorforsyning (f.eks. 9 V fra eksternt batteri)
- **GND:** felles jord for både Arduino og motorforsyning
- **Enable-pinner** (f.eks. EN1, EN2) for å aktivere hver motor
- **Innganger** (f.eks. 1A, 2A) som bestemmer retning
- **Utganger** (f.eks. 1Y, 2Y) koblet til motorens to terminaler

Et typisk tilkoblingseksempel for to motorer:

- 5V og GND fra Arduino kobles til **VCC1** og GND på driverkortet.

- Ekstern motorforsyning (f.eks. 9V og GND fra et batteri) kobles til **VCC2** og samme GND.
- **Motor 1** kobles mellom **1Y** og **2Y**.
- **Motor 2** kobles mellom tilsvarende utganger på den andre siden (f.eks. 3Y og 4Y).
- **Enable1** styrer Motor 1:
  - Settes HIGH for å aktivere motoren.
  - Kan styres med PWM (`analogWrite()`) for å justere hastighet.
- **Enable2** styrer Motor 2 på samme måte.
- Hver motor har to logikk-innganger (f.eks. **1A** og **2A** for Motor 1):
  - HIGH/LOW-kombinasjoner på inngangene bestemmer retningen på motoren.
  - F.eks. **1A = HIGH**, **2A = LOW** gir én retning, mens **1A = LOW**, **2A = HIGH** gir motsatt retning.

## Oppsummering

- **Felles GND** er nesten alltid første punkt å kontrollere når noe ikke virker.
- Kontroller at pinnen i koden faktisk er den som er brukt på breadboardet.
- For «aktive» enheter (servo, NeoPixel, buzzer, små motorer) må det tas ekstra hensyn til strøm og støy.
- For sensorer på I<sup>2</sup>C/SPI handler det ofte om korrekt busskobling, riktig adresse (I<sup>2</sup>C) og korrekt chip select (SPI).

## Seriell kommunikasjon (`Serial.begin` og `baud rate`)

Mange av eksemplene bruker `Serial` for å skrive meldinger til PC-en. To ting må stemme for at teksten i Serial Monitor skal bli lesbar:

- **Samme baud rate i koden og i Arduino IDE**
  - I `setup()` står det ofte noe som:
 

```
Serial.begin(9600);
```
  - Tallet **9600** er *baud rate* (bit per sekund).
  - Nederst til høyre i Serial Monitor må du velge *samme* verdi (f.eks. 9600, 115200 osv.).
  - Hvis baud rate ikke stemmer overens, får du bare «tulleskrift»/rare tegn.
- **Bare én `Serial.begin()` i `setup()`**
  - Sett `Serial.begin(...)`; *én gang* i `setup()`, ikke inne i `loop()`.
  - Endrer du baud rate i koden (f.eks. fra **9600** til **115200**), må du:
    1. lagre og *laste opp* koden på nytt til Arduino
    2. velge samme nye baud rate i Serial Monitor.
- **Riktig port i Arduino IDE**
  - I menyen *Verktøy* → *Port* må riktig COM-/tty-port være valgt (den som tilhører Arduino-en).
  - Hvis feil port er valgt, får du ingen utskrift i det hele tatt (eller feilmelding ved opplasting).
- **Bruk `Serial.print()` aktivt til feilsøking**
  - Skriv ut måleverdier (`analogRead()`, `digitalRead()` osv.) for å se hva som faktisk skjer.
  - Dette er ofte den raskeste måten å finne ut om problemet er i *koden* eller *koblingen*.

## 17 Eksempel på state-maskin

```
/*
Logikk:
- Fire states: GREEN, YELLOW, RED, EMERGENCY
- Normal syklus: 5s grønn 2s gul 5s rød
- Emergency: blinkende gul LED når knapp trykkes
- State machine gjør koden modulær og enkel å utvide
- Ikke-blokkerende timing med millis()
*/

const int red = 9, yellow = 10, green = 11; // Definerer hvilke pins som styrer LED-ene
const int btn = 2;                          // Pin for knappen

enum State // Definerer en enum med navn på alle tilstander
{
    GREEN,
    YELLOW,
    RED,
    EMERGENCY
};

State state = GREEN; // Starttilstanden er GREEN
unsigned long lastChange = 0; // Variabel for å lagre tidspunktet siste state-endring skjedde

void setup()
{
    pinMode(red, OUTPUT); // Setter rød LED-pin som output
    pinMode(yellow, OUTPUT); // Setter gul LED-pin som output
    pinMode(green, OUTPUT); // Setter grønn LED-pin som output
    pinMode(btn, INPUT_PULLUP); // Knapp med intern pull-up, aktiv LOW
}

void loop()
{
    bool emergency = digitalRead(btn) == LOW; // Leser knappen: LOW betyr trykket inn (emergency)

    if (emergency) // Hvis knapp trykkes
        state = EMERGENCY; // Sett state til EMERGENCY umiddelbart

    switch (state) // Velg logikk basert på nåværende state
    {
        case GREEN: // Tilstand: GRØNN
            digitalWrite(green, HIGH); // Skru på grønn LED
            digitalWrite(yellow, LOW); // Skru av gul LED
            digitalWrite(red, LOW); // Skru av rød LED
            if (millis() - lastChange > 5000) // Har 5 sekunder gått?
            {
                state = YELLOW; // Gå til YELLOW
                lastChange = millis(); // Oppdater tidspunktet
            }
            break;

        case YELLOW: // Tilstand: GUL
            digitalWrite(green, LOW); // Skru av grønn
            digitalWrite(yellow, HIGH); // Skru på gul
            digitalWrite(red, LOW); // Skru av rød
            if (millis() - lastChange > 2000) // Har 2 sekunder gått?
            {
                state = RED; // Gå til RØD
                lastChange = millis(); // Oppdater tidspunkt
            }
            break;

        case RED: // Tilstand: RØD
            digitalWrite(green, LOW); // Skru av grønn
```

```

digitalWrite(yellow, LOW);          // Skru av gul
digitalWrite(red, HIGH);             // Skru på rød
if (millis() - lastChange > 5000) // Har 5 sekunder gått?
{
    state = GREEN;                  // Gå til GRØNN
    lastChange = millis(); // Oppdater tidspunkt
}
break;

case EMERGENCY:                      // Tilstand: EMERGENCY
    digitalWrite(green, LOW);        // Skru av grønn
    digitalWrite(yellow, millis() % 500 < 250 ? HIGH : LOW); // Blink gul: 250ms på, 250ms av
    digitalWrite(red, LOW);          // Skru av rød
    break;
}
}

```